

EC-467 Mobile Application Development
BS (CE)-2021
Complex Engineering problem



Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz

Submitted To:

Sir Dr. Muhammad Bilal

Department of Computer Engineering

HITEC University Taxila

Complex Engineering problem

A school management required you to design a “Student Management System” for android mobile to manage student’s record. They want you to maintain student’s registration number, name, parent phone number, and address, and the management can search the students by their name or registration number and display list of student. The school management is not sure will the “Cloud” based system will be more appropriate or locally stored management system. Develop two an android application one application store student’s record locally (using SQLite) and other cloud based (using Firestore database). Now compare both application in- term of performance, security and usability.

SQLite database:

JAVA CODE OF MAIN ACTIVITY:

```
package com.huzaifashahbaz.sqlitedatabaseapp;
import android.database.Cursor;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ListView;
import android.widget.Toast;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import java.util.ArrayList;
public class MainActivity extends AppCompatActivity {
    Button btnAddData, btnViewAll;
    DatabaseHelper myDb;
    EditText editName, editRegNumber, editParentPhone, editAddress;
    ListView listView;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
```

```

super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
myDb = new DatabaseHelper(this);
editName = findViewById(R.id.editTextName);
editRegNumber = findViewById(R.id.editTextRegNumber);
editParentPhone = findViewById(R.id.editTextParentPhone);
editAddress = findViewById(R.id.editTextAddress);
listView = findViewById(R.id.listview);
btnAddData = findViewById(R.id.button_add);
btnViewAll = findViewById(R.id.button_view_all);
btnAddData.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        addData();
    }
});
btnViewAll.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        viewAll();
    }
});
}

public void addData() {
    boolean isInserted = myDb.insertData(
        editRegNumber.getText().toString(),
        editName.getText().toString(),
        editParentPhone.getText().toString(),
        editAddress.getText().toString()
    );
    if (isInserted) {
        Toast.makeText(MainActivity.this, "Data Inserted",
Toast.LENGTH_LONG).show();
    } else {
        Toast.makeText(MainActivity.this, "Data not inserted",
Toast.LENGTH_LONG).show();
    }
}

public void viewAll() {
    Cursor res = myDb.searchByName(editName.getText().toString());

```

```

        if (res.getCount() == 0) {
            showMessage("Error", "Nothing found");
            return;
        }
        ArrayList<String> list = new ArrayList<>();
        while (res.moveToNext()) {
            list.add("Registration Number: " + res.getString(0) + "\n" +
                "Name: " + res.getString(1) + "\n" +
                "Parent Phone: " + res.getString(2) + "\n" +
                "Address: " + res.getString(3));
        }
        ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
            android.R.layout.simple_list_item_1, list);
        listView.setAdapter(adapter);
    }
    public void showMessage(String title, String message) {
        AlertDialog.Builder builder = new AlertDialog.Builder(this);
        builder.setCancelable(true);
        builder.setTitle(title);
        builder.setMessage(message);
        builder.show();
    }
}

```

JAVA CODE OF DATABASE HELPER:

```

package com.huzaifashahbaz.sqlitedatabaseapp;
import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
public class DatabaseHelper extends SQLiteOpenHelper {
    private static final String DATABASE_NAME="student.db";
    private static final String TABLE_NAME="student_table";
    private static final String COL_1="REGISTRATION_NUMBER";
    private static final String COL_2="NAME";
    private static final String COL_3="PARENT_PHONE";
    private static final String COL_4="ADDRESS";
    public DatabaseHelper(Context context)
    {

```

```

        super(context,DATABASE_NAME,null,1);
    }
    @Override
    public void onCreate(SQLiteDatabase db)
    {
        db.execSQL("CREATE TABLE " + TABLE_NAME +
"(REGISTRATION_NUMBER INTEGER PRIMARY KEY,NAME
TEXT,PARENT_PHONE TEXT,ADDRESS TEXT)");
    }
    @Override
    public void onUpgrade(SQLiteDatabase db,int oldVersion,int newVersion)
    {
        db.execSQL("DROP TABLE IF EXISTS "+ TABLE_NAME);
        onCreate(db);
    }
    public boolean insertData(String regNumber,String name,String
parentPhone,String address)
    {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
        contentValues.put(COL_1, regNumber);
        contentValues.put(COL_2, name);
        contentValues.put(COL_3, parentPhone);
        contentValues.put(COL_4, address);
        long result = db.insert(TABLE_NAME, null, contentValues);
        return result != -1;
    }
    public Cursor searchByNme(String name)
    {
        SQLiteDatabase db=this.getWritableDatabase();
        return db.rawQuery("SELECT * FROM " + TABLE_NAME + " WHERE
NAME LIKE ?", new String[]{"%" + name + "%"});
    }
    public Cursor searchByRegNumber(String regNumber)
    {
        SQLiteDatabase db = this.getWritableDatabase();
        return db.rawQuery("SELECT * FROM " + TABLE_NAME + " WHERE
REGISTRATION_NUMBER = ?", new String[]{regNumber});
    }
}

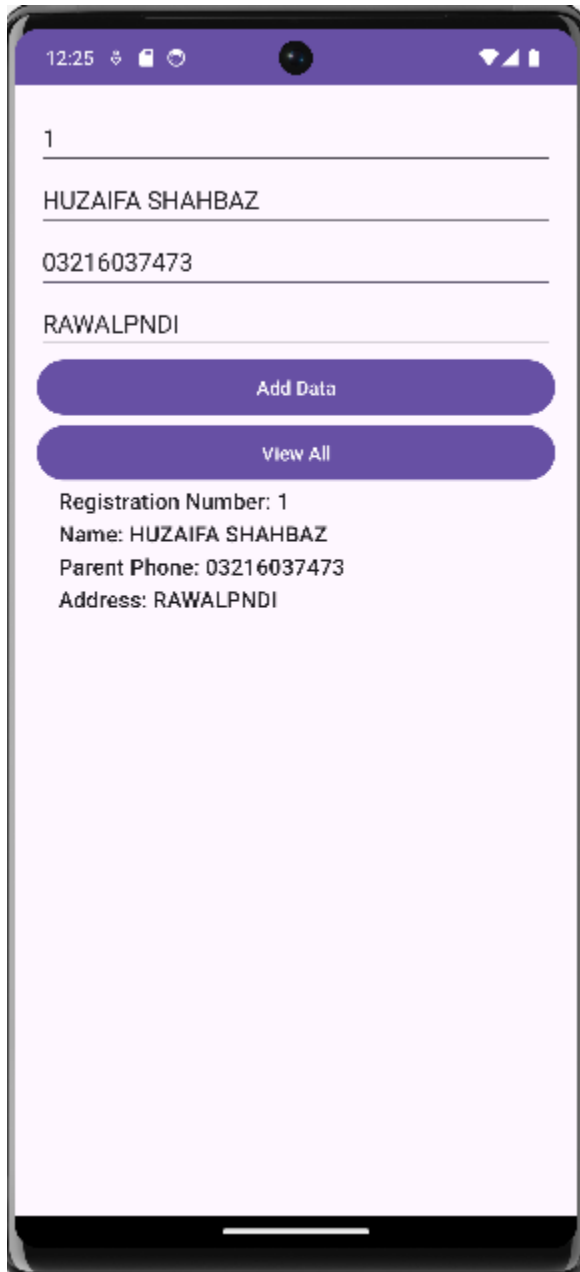
```

XML CODE OF MAIN ACTIVITY:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">
    <EditText
        android:id="@+id/editTextRegNumber"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Registration Number"/>
    <EditText
        android:id="@+id/editTextName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Name"/>
    <EditText
        android:id="@+id/editTextParentPhone"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Parent Phone"/>
    <EditText
        android:id="@+id/editTextAddress"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Address"/>
    <Button
        android:id="@+id/button_add"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:onClick="onPressButton"
        android:text="Add Data"/>
    <Button
        android:id="@+id/button_view_all"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
```

```
        android:onClick="onPressButton"
        android:text="View All"/>
<ListView
    android:id="@+id/listview"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>
</LinearLayout>
```

Results:



FIRESTORE DATABASE

JAVA CODE OF MAIN ACTIVITY:

```
package com.huzaifashahbaz.firestoredatabaseapp;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import com.google.firebase.firestore.DocumentReference;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.FirebaseFirestoreException;
import com.google.firebase.firestore.Transaction;
public class MainActivity extends AppCompatActivity {
    private EditText regNumberEditText, nameEditText, parentPhoneEditText,
addressEditText;
    private Button saveButton;
    private FirebaseFirestore db;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        db = FirebaseFirestore.getInstance();
        regNumberEditText = findViewById(R.id.editTextRegNumber);
        nameEditText = findViewById(R.id.editTextName);
        parentPhoneEditText = findViewById(R.id.editTextParentPhone);
        addressEditText = findViewById(R.id.editTextAddress);
        saveButton = findViewById(R.id.buttonSave);
        saveButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                saveStudent();
            }
        });
    }
    private void saveStudent() {
        String regNumber = regNumberEditText.getText().toString();
        String name = nameEditText.getText().toString();
```



```

String parentPhone = parentPhoneEditText.getText().toString();
String address = addressEditText.getText().toString();
if (regNumber.isEmpty() || name.isEmpty() || parentPhone.isEmpty() ||
address.isEmpty()) {
    Toast.makeText(this, "Please fill all fields",
Toast.LENGTH_SHORT).show();
    return;
}
Student student = new Student(regNumber, name, parentPhone, address);
db.collection("students").document(regNumber).set(student)
    .addOnSuccessListener(aVoid -> {
        Toast.makeText(MainActivity.this, "Student saved successfully",
Toast.LENGTH_SHORT).show();
        clearFields();
    })
    .addOnFailureListener(e -> Toast.makeText(MainActivity.this, "Error
saving student: " + e.getMessage(), Toast.LENGTH_SHORT).show());
}
private void clearFields() {
    regNumberEditText.setText("");
    nameEditText.setText("");
    parentPhoneEditText.setText("");
    addressEditText.setText("");
}
}

```

JAVA CODE OF STUDENT:

```

package com.huzaifashahbaz.firestoredatabaseapp;
public class Student {
    private String registrationNumber;
    private String name;
    private String parentPhone;
    private String address;
    public Student() {}
    public Student(String registrationNumber, String name, String parentPhone,
String address) {
        this.registrationNumber = registrationNumber;
        this.name = name;
        this.parentPhone = parentPhone;
        this.address = address;
    }
}

```

```

    }
    public String getRegistrationNumber() {
        return registrationNumber;
    }
    public void setRegistrationNumber(String registrationNumber) {
        this.registrationNumber = registrationNumber;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getParentPhone() {
        return parentPhone;
    }
    public void setParentPhone(String parentPhone) {
        this.parentPhone = parentPhone;
    }
    public String getAddress() {
        return address;
    }
    public void setAddress(String address) {
        this.address = address;
    }
}

```

XML CODE OF MAIN ACTIVITY:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">
    <EditText
        android:id="@+id/editTextRegNumber"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Registration Number"
        android:inputType="text"
    >

```

```
        android:padding="10dp"
        android:layout_marginBottom="8dp"/>
<EditText
    android:id="@+id/editTextName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Name"
    android:inputType="text"
    android:padding="10dp"
    android:layout_marginBottom="8dp"/>
<EditText
    android:id="@+id/editTextParentPhone"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Parent Phone"
    android:inputType="phone"
    android:padding="10dp"
    android:layout_marginBottom="8dp"/>
<EditText
    android:id="@+id/editTextAddress"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Address"
    android:inputType="textPostalAddress"
    android:padding="10dp"
    android:layout_marginBottom="16dp"/>
<Button
    android:id="@+id/button_add"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Add Data"
    android:padding="10dp"
    android:onClick="onPressButton"
    android:layout_marginBottom="8dp"/>
<Button
    android:id="@+id/button_view_all"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="View All"
    android:padding="10dp"
```

```

        android:onClick="onPressButton"
        android:layout_marginBottom="16dp"/>
<ListView
    android:id="@+id/listview"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:divider="@android:color/darker_gray"
    android:dividerHeight="1dp"/>
</LinearLayout>

```

DIFFERENCES:

Criteria	SQLite	Firestore
Performance	Pros: - Faster for local read/write operations since data is stored locally	Pros: - Scales well with large datasets. - Better for real-time updates and sync across devices.
	Cons: - Limited by device storage. - Can slow down if handling large datasets.	Cons: - Dependent on network connectivity. - Latency might be an issue for some operations.
Security	Pros: - Data is stored locally, which can be more secure if the device is secure.	Pros: - Data is stored securely in the cloud with encryption at rest and in transit. - Firebase Authentication can be used to secure data access
	Cons: - Vulnerable to local attacks if the device is compromised.	Cons: - Reliant on the security of the cloud provider. - Requires proper rules setup to ensure data protection.
Usability	Pros: - Simple to implement for basic local storage needs. - No internet required.	Pros: - Supports real-time updates. - Cross-device sync. - Offline capabilities. -

		Easier to scale and manage remotely.
	Cons: - Limited scalability. - Does not support real-time updates or cross-device synchronization.	Cons: - Requires internet connection for initial setup and synchronization. - Can be more complex to implement and manage.